
Assignement 2 : Oxono AI agent

1 Description

The objective of this project is to design an AI agent capable of playing Oxono effectively. As a deterministic, two-player, zero-sum game with perfect information, Oxono lends itself to adversarial search techniques[1].

Our approach is based on the Alpha-Beta pruning algorithm, an optimized version of the Minimax algorithm[2], combined with a heuristic evaluation function to assess non-terminal states. The initial implementation followed the classical structure described in *Artificial Intelligence: A Modern Approach (AIMA)*[2], using a fixed-depth search and a simple handcrafted heuristic based on piece alignments.

From this baseline, we progressively enhanced both the search procedure and the evaluation function to improve efficiency and tactical awareness. In particular, we introduced iterative deepening with time management, a transposition table based on Zobrist hashing, and improved move ordering strategies. Additional refinements include quiescence search to mitigate the horizon effect and explicit detection of immediate winning or losing moves. These improvements allow the agent to search deeper within the same time constraints and to better handle tactical situations.

Improvements to the Baseline Algorithms

We first introduced iterative deepening to allow the agent to explore the search space incrementally[2]. In early stages of the game, the search remains shallow, while in later stages it becomes deeper as the board becomes more constrained. This ensures that a valid move is always available and improves decision quality under varying conditions.

A time management policy was then integrated with iterative deepening. This policy relies on both soft and hard time limits, with time per move allocated dynamically according to a formula inspired by existing research[5]. Although the implementation is functional, it was observed that time is not always optimally distributed, with excess time often remaining unused at the end of games.

To reduce redundant computations, a transposition table based on Zobrist hashing was implemented[1]. The first version used a standard dictionary, while a second version introduced an LRU cache structure to control memory usage and ensure constant-time access. Each entry stores not only evaluation values but also bound information (exact, lower, or upper), allowing better integration with Alpha-Beta pruning. Hash computation was further optimized by using incremental XOR updates instead of recomputing the entire state hash[1], reducing overhead during search. Key collision may occur with Zobrist hashing, but they are extremely rare. In our case, with a transposition table size of 100k and 64-bit hash values, this issue can be neglected,

and no additional validity checks are required (except when using the stored move for move ordering).

Move ordering was also introduced to improve pruning efficiency[1]. Moves retrieved from the transposition table are prioritized when reliable, while the remaining moves are evaluated heuristically and sorted accordingly. This significantly increases the effectiveness of Alpha-Beta by exploring promising branches first.

To enhance tactical play, we added a mechanism to detect immediate winning moves. Whenever a move leads to a direct win, it is selected without further search. This effectively acts as a local depth extension and prevents the agent from missing trivial victories.

Finally, quiescence search was implemented to mitigate the horizon effect[1]. By extending the search in unstable positions, the agent avoids evaluating states prematurely. However, the performance of this technique proved sensitive to parameter tuning, and its effectiveness depends heavily on the quality of the evaluation function.

Experiments

Throughout development, multiple configurations of the agent were tested in order to assess the impact of each improvement. Because most enhancements are interdependent, isolating their individual contributions proved challenging.

Nevertheless, several general trends were observed. Iterative deepening improved robustness under time constraints, while move ordering and transposition tables significantly enhanced search efficiency. Quiescence search showed benefits in tactical situations but required careful tuning. It was also observed that improvements to the evaluation function often had a greater impact on performance than increasing search depth alone.

Other Attempts

We experimented with Monte Carlo Tree Search (MCTS)[1] as an alternative to Alpha-Beta. However, due to the highly tactical nature of Oxono, MCTS performed significantly worse. Its reliance on random simulations makes it less effective at detecting short-term forced sequences, which are critical in this game.

Value Network

We also explored a learning-based approach by introducing a neural network as a value function. Early experiments quickly highlighted the importance of the evaluation function, as the overall performance of the agent is highly sensitive to the quality of state evaluation.

A convolutional neural network (CNN) was selected instead of a purely dense architecture, as it preserves the spatial structure of the board and captures local interactions between pieces[6]. This choice is consistent with common practices in board game AI, where spatial correlations play a central role.

The model was built on tensorflow as we already were familiar with this library and was trained on a dataset of approximately 50,000 games, corresponding to around 900,000 states, generated from Alpha-Beta agents and self-play. To improve training quality, we applied sample weighting to reduce the influence of early-game states and assigned labels based on final game outcomes with a discount factor (TD(0) discounted), this factor was set arbitrarily to 0.95 as most game in the datasets were 17 moves long so that the early game evaluation would be more than half the end game utility.

The value network was build on a modular and fully parametrisable architecture where each hyperparameter could be fine tuned. To help us winding down our research for the optimal architecture we used assumption based on our reading of others projects and paper. The activation function was set to ReLu as it's mainly used across all projects the output node used the tanh as it give an output bounded between $[-1,1]$ also common practices. The architecture of n-CNN layer followed by a dense layer and an output node is based also on common practices in value network.

Hyperparameter optimization was performed using Optuna, a Bayesian optimization framework designed for pytorch that proved effective and easy to use compared to built-in tensorflow optimizers. It also provides useful visualization tools that helped guide our analysis.

Figure 1 shows the evolution of the objective value across trials. The model converges rapidly during the first iterations, followed by a slower refinement phase. A few outliers are observed early on, but after approximately 150 trials, the optimization stabilizes and no significant improvements are found, likely due to both convergence and time constraints.

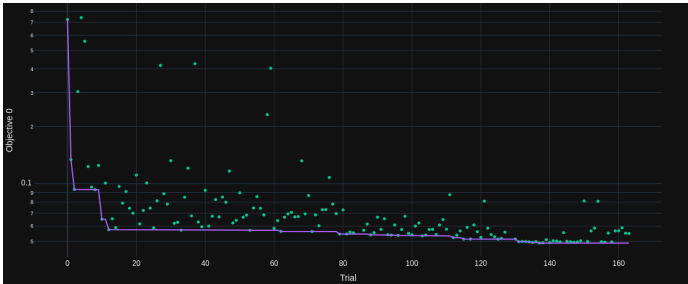


Figure 1: Optuna optimization history showing objective value across trials

Figure 2 presents the importance of each hyperparameter. The Huber loss parameter (`huber_delta`) appears to be the most influential. This is consistent with its role in balancing between mean squared error (MSE) and mean absolute error (MAE), allowing the model to handle both small and large errors more effectively[7]. Other parameters, such as the optimizer choice, appear to have a relatively minor impact.

Optuna identified the following optimal hyperparameters: learning rate $lr = 2.78 \times 10^{-4}$, Adam optimizer, Huber loss with $\delta = 0.31$, a convolutional architecture with 2 layers and 121 filters, a dense layer of size 256, and a batch size of 74.

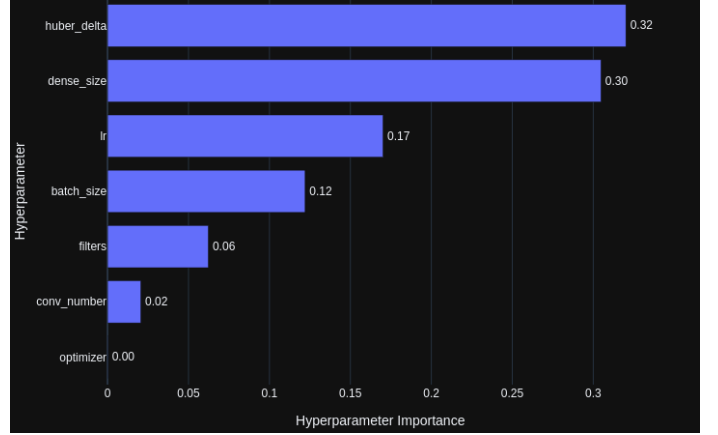


Figure 2: Hyperparameter importance estimated by Optuna

Based on these results, we selected the best-performing hyperparameter configuration found by Optuna. Additionally, some parameters such as batch size and number of filters were constrained to powers of two, as commonly recommended for computational efficiency and hardware optimization.

Finally, Figure 3 provides a parallel coordinate plot of the explored hyperparameter space. While it does not directly inform design decisions, it offers a global view of the search process and illustrates the relationship between parameters and performance.

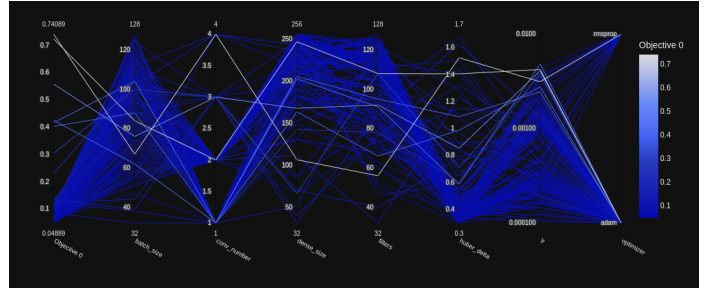


Figure 3: Parallel coordinate plot of hyperparameter configurations explored by Optuna

Further work is needed to stabilize training and improve its impact on gameplay. In particular, we considered implementing temporal-difference learning methods such as $TD(\lambda)$ [2], but did not complete this due to the difficulty of tuning its parameters. Additional techniques, such as ProbCut a forward pruning algorithm [9], were also considered but not implemented within the scope of the project.

Design Justification

The choice of Alpha-Beta search is strongly motivated by the structural properties of Oxono. In such settings, Alpha-Beta search is particularly effective, as it enables the systematic exploration of the game tree while pruning large portions of

the search space, allowing the detection of forced wins and tactical patterns.

Iterative deepening combined with time management allows the agent to adapt its search depth dynamically based on the remaining time and game phase. A transposition table based on Zobrist hashing is used to avoid redundant computations and reuse previously evaluated positions. Additionally, enhanced move ordering strategies, leveraging heuristic evaluation, transposition table information, and threat detection significantly improve pruning efficiency.

The agent also detects immediate winning and losing moves and uses a threat-sensitive evaluation function to prioritize critical positions.

In contrast, Monte Carlo Tree Search relies on stochastic simulations and typically requires a large number of rollouts to approximate the value of a position. While effective in high-branching environments with long-term strategic considerations, we found it to be less reliable in games dominated by precise tactical sequences.

Although learning-based approaches and neural-networks were explored, they were not integrated into the final agent due to stability and performance limitations. Instead, we focused on refining the deterministic search framework, which proved to be more effective within the given computational constraints.

2 Evaluation

Experimental Setup

To evaluate the performance of our final agent, we conducted a series of experiments using an automated benchmarking script. We first conducted an ablation study by adding individual components to isolate their contribution to performance. Then we evaluated our final agent (`my_agent.py`) against three reference opponents of different strengths: a random agent, a baseline Alpha-Beta agent, and a baseline Monte Carlo Tree Search (MCTS) agent. For each matchup, a total of 20 games were played, after 10 games players are switched to measure any first-move advantage. Each game was executed under identical conditions, ensuring fairness and reproducibility. The evaluation focuses on performance metrics, including win rate, average game length, and average computation time per move. These metrics allow us to assess the strength and efficiency of the agent. The results are aggregated over multiple runs for analysis and visualization.

Performance

For the ablation study, we tested different version of our agent against each other, the version are *AB-time adaptive*, *AB-time adaptive + transposition table* and *AB-time adaptive + transposition table + move ordering + threat detection* (our final agent).

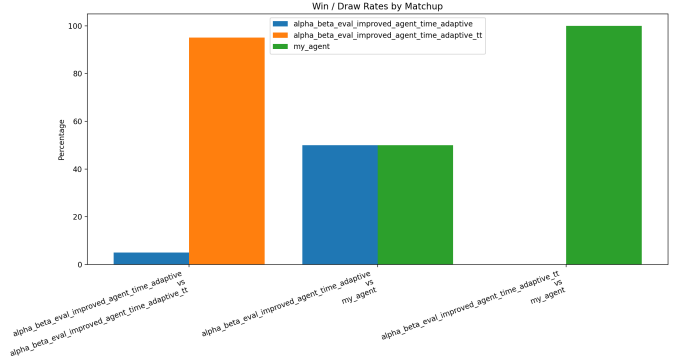


Figure 4: Win rate by matchup

As shown in Figure 4, *my_agent* outperforms both baselines, achieving a 100% win rate against *AB-time adaptive* and a 50/50 split against *AB-time adaptive + transposition table*.

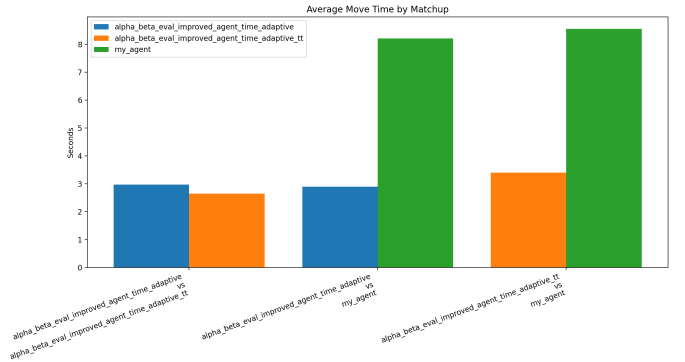


Figure 5: Average move time by matchup

my_agent incurs higher move times than the ablated versions, indicating that the added optimizations improve decision quality at the cost of computational overhead.

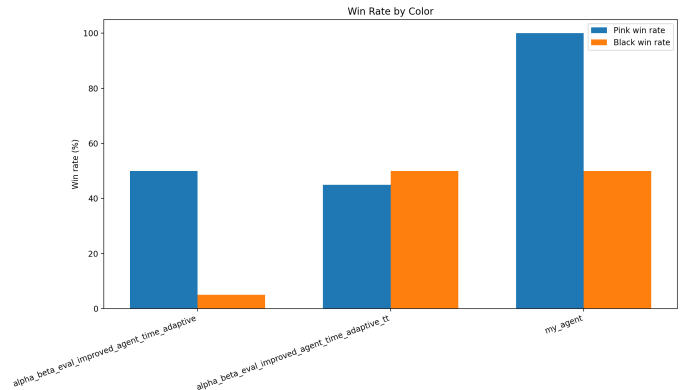


Figure 6: Win rate by color

In Figure 6, it can be seen that across all variants, the first player (pink) holds a clear advantage, most pronounced in the final agent where it achieves a 100% win rate.

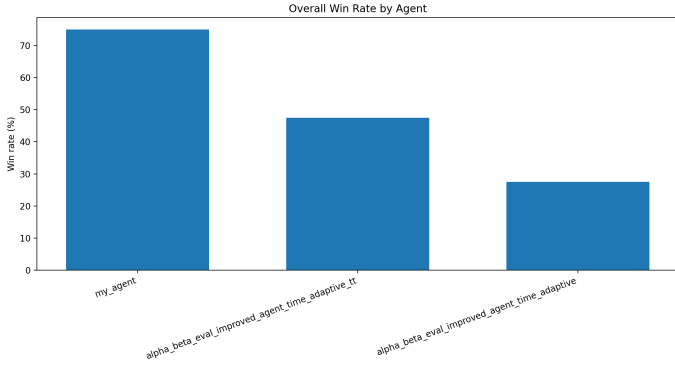


Figure 7: Overall win rate by agent

Figure 7 shows that the final agent achieves the highest overall win rate, substantially outperforming both ablated versions.

Comparison with Reference Agents

The experimental results show that the proposed agent performs well across all tested matchups. As illustrated in Figure 8, our agent consistently wins all games against the random agent.

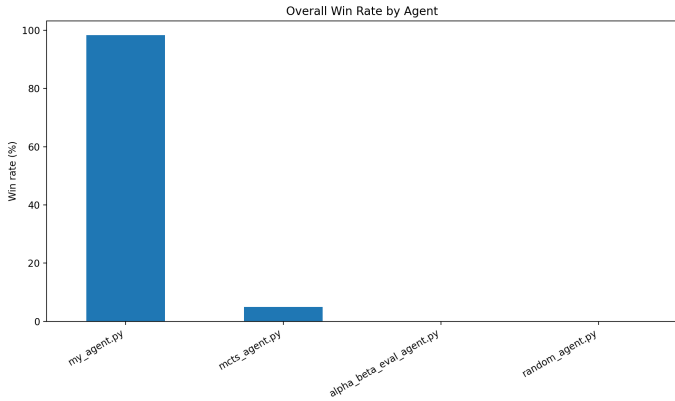


Figure 8: Overall win rate by agent

The agent achieves a perfect win rate against the baseline Alpha-Beta agent for the 20 games played, indicating that the implemented optimizations enhance the efficiency of the search.

When facing the MCTS agent, the performance decreases but remains dominant, with only one game lost on 20. This suggests that while stochastic exploration can occasionally discover strong moves.

As shown in Figure 9, the average move time is significantly higher than that of simpler agents, reaching several seconds per move. The baseline Alpha-Beta agent exhibits higher computation times in some matchups, showing that the improved pruning and ordering strategies reduce unnecessary exploration while maintaining strong performance.

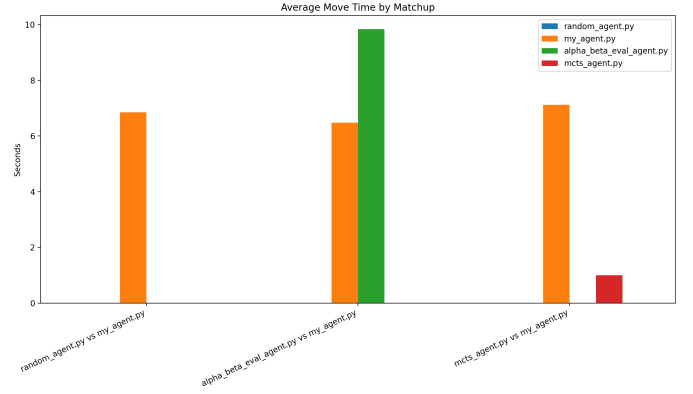


Figure 9: Average move time by matchup

Game length analysis (Figure 10) indicates that matches against stronger opponents tend to last longer, reflecting more balanced and strategic play. In contrast, games against the random agent are shorter, as the agent quickly exploits suboptimal moves.

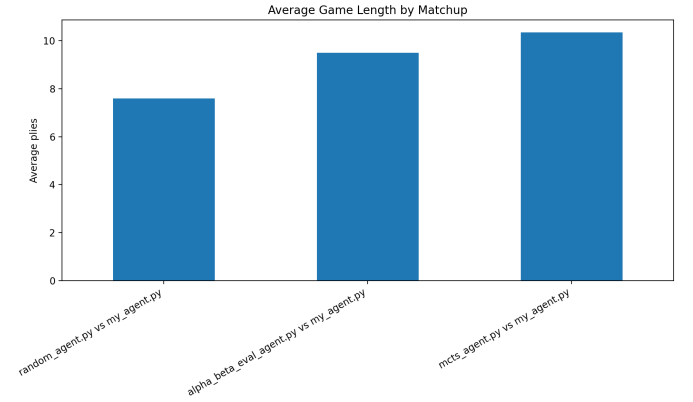


Figure 10: Average plies by matchup

Finally, the results by color (Figure 11) show that the agent performs consistently well as both first and second player. Although a slight advantage is observed when playing first, we cannot conclude as the test set of only 10 games per color is not large enough.

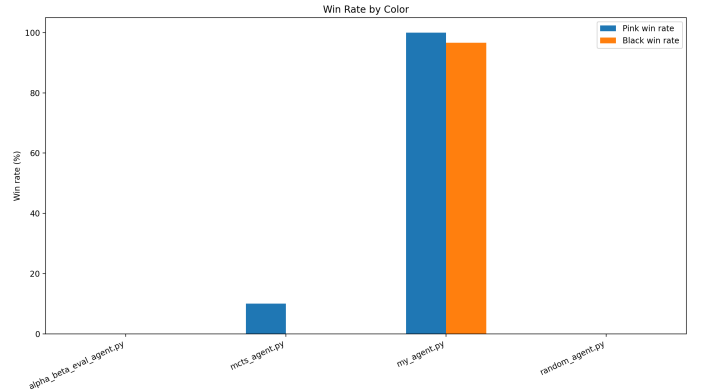


Figure 11: Win rate by color

Overall, the results confirm that an optimized Alpha-Beta agent with transposition tables, better move ordering, and adaptive time management remains highly competitive

in structured deterministic game like Oxono where tactical precision is critical.

3 Discussion

The results confirm that Alpha-Beta search is well-suited for Oxono, a game dominated by short-term tactical interactions. Deterministic search allows the agent to reliably detect forced wins and avoid immediate threats, which explains its strong performance.

The main improvements come from practical optimizations rather than changes to the core algorithm. Iterative deepening with time management enables deeper search, while transposition tables and move ordering reduce the number of explored states. These enhancements significantly improve efficiency and decision quality.

Comparisons with other agents support these observations. The agent consistently outperforms the random and baseline Alpha-Beta agents, and also shows stronger performance than MCTS, whose stochastic nature is less effective in a highly tactical setting.

The time policy enforcement is still not perfect. It mostly times out at the hard limit, but it works for our use case.

Despite these results, the agent remains limited by the quality of its heuristic and search depth. Horizon effects and imperfect evaluations can still lead to suboptimal moves. A promising direction for future work is to integrate reinforcement learning to learn a value function through self-play, potentially improving evaluation accuracy while retaining efficient search.

Although the model showed promising behavior, increasing architectural complexity did not significantly improve performance. The current model is computationally heavy, as it is composed of more than one million weights.

Another issue comes from the TensorFlow overhead and the frequent calls to the evaluation function. A better integration of the CNN is needed, as the current naive approach of directly using it within the evaluation function is too computationally expensive.

An alternative model architecture is under development, using only one convolutional layer and reducing the number of filters to 64 or fewer. This should significantly decrease the computation required per evaluation. These choices are supported by the diagram, which suggests that these parameters have relatively minor importance. Any potential loss in performance could be compensated by improved and longer training.

References

[1] Prof. Éric Piette, *LINFO1361 - Adversarial Search & Games Slides*, École Polytechnique de Louvain, 2026.

[2] Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Chapter 6, 2021.

[3] ChessProgramming Wiki, *ChessProgramming.org*, <http://www.chessprogramming.org>, accessed: 2026.

[4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. Available: <http://www.deeplearningbook.org>

[5] S.-C. Huang, R. Coulom, and S.-S. Lin, “Time management for Monte-Carlo tree search applied to the game of Go,” in *Proceedings of the International Conference on Technologies and Applications of Artificial Intelligence (TAAI)*, 2010, pp. 462–466. Available: <https://www.remi-coulom.fr/Publications/TimeManagement.pdf>

[6] P. Liskowski, W. Jaśkowski, and K. Krawiec, “Learning to Play Othello with Deep Neural Networks,” *IEEE Transactions on Computational Intelligence and AI in Games*, 2018. Available: <https://arxiv.org/abs/1711.06583>

[7] Y. Dave, “A Complete Guide to Deep Learning Loss Functions: MSE, MAE, Huber and Cross-Entropy Explained,” *Medium*, 2023. Available: <https://medium.com/@yashdave9706/a-complete-guide-to-deep-learning-loss-functions-mse-mae-huber-and-cross-entropy-explained-2685b71998e8>

[8] C. Clark and A. Storkey, “Teaching Deep Convolutional Neural Networks to Play Go,” *arXiv preprint arXiv:1412.3409*, 2014. Available: <https://arxiv.org/abs/1412.3409>

[9] E. P. Manning, “Temporal Difference Learning of an Othello Evaluation Function for a Small Neural Network with Shared Weights,” in *Proceedings of the IEEE Symposium on Computational Intelligence and Games (CIG)*, 2007.

4 AI and Sources Acknowledgement

During this project, commercial large language models (LLMs) were used to support and accelerate certain aspects of development. No AI-generated code was used in the design or implementation of the core agent algorithms or the neural network architecture.

LLMs were primarily used to assist with auxiliary tasks such as dataset generation, benchmarking scripts, and text reformulation. They were also used to help debug errors and clarify concepts encountered in the literature.

All core algorithms and agent implementations were developed independently and were inspired by the cited academic sources. Additional informal resources, such as online forums (e.g., Stack Overflow and Reddit), were occasionally consulted for specific technical issues; however, these are not formally cited for conciseness.